

Build for Billions: Initial Screening Proposal

LEGISIM, a flight simulator for policies. Because a law should not be test flown on real people first.

Team name	neuro-cooked
Team leader	Dikshit Rishi Jain
Team members	Vaishnavi Saraf, Sowmiyanathan Raja, Sayan Maity, Shreya Devendra
College/University	National Institute of Technology Karnataka
Selected track	Agentic AI For Billions-Autonomous workflows for public and social services.

1. Problem Statement

Fuel gets Rs 10 a litre costlier. Somewhere, a tomato gets more expensive, and nobody mentioned it in the press release.

Every policy arrives with a one line promise and a long list of surprises. Take that fuel price rise. Truckers pay more first. Freight costs go up, so vegetables cost more, so household budgets tighten. People also travel less, so shops see fewer visitors. The announcement covered none of this.

Nearly everyone faces this problem, but not equally. The same policy lands differently on a farmer in Vidarbha and a software engineer in Bengaluru. Today, the answer to "who gets hit, and when?" sits scattered across think tank reports and expert opinion. Students, journalists and policy teams either wait weeks for a deep study or guess. A wrong guess about a policy that touches over a billion people is expensive, which is why a fast, explainable first estimate matters.

2. Proposed Solution

We put a whole country in a browser tab, then asked it what happens when fuel gets costlier.

LEGISIM lets you customise a country (India first), introduce a policy, run the simulation and watch different groups of people respond in their own way. It tracks how those reactions add up across the economy and society over time.

How it works: we do not simulate a billion individual people (our laptops would file a complaint). We simulate a few hundred to a few thousand weighted groups, called cohorts, such as urban salaried households or small farmers in one region. Each cohort decides how to change its spending, work and travel, using real data, simple economic models and what happened after similar past policies. AI handles the judgement calls and the explanations, and the numbers come from data. A ripple engine then pushes the first change through a cause and effect map, step by step.

Example run: fuel price up by Rs 10 a litre. Freight cost rises first, food prices follow, and household budgets absorb the rest. LEGISIM shows each hop, how big it is, how long it takes and how sure we are.

Features we plan to build:

- Ripple explorer with a Play button that animates the chain from policy to prices to household budgets, with filters for group, topic, depth and time.
- Time slider for short and long term effects, including how long each group takes to accept a change.
- Impact dashboard and India map, with sliders for income, region, urban or rural, education and duration.
- A plain language summary report beside the charts, written like a short research note.
- An evidence drawer on every number, showing the reasoning, data and sources, plus a label saying whether the number was measured, modelled or judged.

3. Target Users & Impact

Winners, losers, and the people the national average forgot about.

Our users are government researchers and analysts who need a quick and explainable first estimate, policy teams and think tanks comparing options before a deep study, and curious citizens and teachers.

Policy debates usually talk about averages, and averages hide people. LEGISIM gives each group its own row, so a low income rural household does not get averaged away by a metro salary. The winners and losers view puts the first question a policy reader asks on the first screen. We also plan a Hindi and Kannada interface, lakh and crore number formats, and a guest mode so anyone with a browser can try a scenario without signing up. We do not claim a crystal ball: every output shows its assumptions, sources and confidence, so users can judge it themselves.

4. Technology Stack

The nerdy part, in plain words.

- **Website:** Next.js with TypeScript with Tailwind CSS and shadcn/ui. Charts use Apache ECharts, the ripple graph uses React Flow, and the India map uses MapLibre GL with deck.gl.
- **Backend:** Python with FastAPI. LangGraph runs the whole pipeline (research, cohort reactions, the economic step, the ripple engine, the report, and a critic check that reviews the report before anyone sees it) and streams live progress to the screen.
- **AI models:** two tiers. A stronger model plans, proposes ripple links and writes reports. A small, fast model handles the many cohort reactions. Outputs use structured JSON, and model names live in one config file so we can swap them easily.
- **Numbers and data:** NumPy, pandas and SciPy for the economic model and uncertainty runs. PostgreSQL with pgvector stores results and the past policy library, Redis runs background jobs, and official government data downloads feed the population and economy tables.
- **Around it:** MongoDB for database, LangGraph to trace every AI call, Pytest and Playwright for testing, and Docker for deployment.

5. Innovation & Scalability

Plenty of chatbots can write about policy. Ours shows its homework.

Ripple effects are the headline feature, so they get their own engine instead of free form AI writing. A hand built map of trusted links (fuel to freight to food prices) forms the base. The AI can propose extra links, but each one must state its mechanism in one sentence and point to a source or a past example, or it gets dropped or marked low confidence.

Predictions also look backward. A historical analogue finder can say "this looks like 2016 demonetisation", and a backtest mode reruns a past policy with the data cut off to compare against what actually happened. Repeated runs give every number a confidence range.

To scale, we run thousands of groups rather than millions of AI agents, which keeps cost and time sensible. Results are precomputed into a small data cube, so the sliders respond instantly with no new AI calls. New policies arrive as templates (fuel price, tax slab, farm loan waiver, minimum wage, EV subsidy, free bus travel), and the country and population settings are customisable, so India comes first and other countries can follow. District level drill down is a later step.

6. Prototype / References

Yes, it exists. It is honest about being a demo.

Pitchdeck: <https://canva.link/63eckd15hd8d5tu>

Demo video: <https://youtu.be/9Fs-eZ6rutg> (Trust us, the video is pretty)

7. Team Contribution

Five people, one simulated India.

Dikshit Rishi Jain (leader)	Backend and AI: the LangGraph pipeline, cohort builder, economic model, ripple engine, API and database.
Vaishnavi Saraf	Data collection lead: gathering and cleaning data, population tables and the past policy library.
Sowmiyanathan Raja	Frontend and integration: results layout, shared state, live progress streaming and the dashboard sliders.
Sayan Maity	Frontend, ideation and design: the design system, UI pages, ripple graph and map visuals, and the demo story.
Shreya Devendra	Evidence and validation: backtests, report review and the methodology page.